



Rectitation 7

Instructions for tasks

- Copy `/share/recitations/recitation07/bitPositions.c` to your directory.
- We implemented the first three functions **getLSBpos**, **getMSBpos**, **getNumPos**) before Pessach.
- Now we implement the remaining two functions: **getNextBitPos** **printBP**.
- Copy `/share/recitations/recitation07/test_bitPositions.c` to your directory. This file contains a single main function that runs tests for all functions.
- To generate a program that runs the execution examples for task X, compile as follows:

```
gcc -Wall bitPositions.c test_bitPositions.c -D TEST_X -o test_X
```
- The `-D TEST_X` 'turns on' the code that is wrapped with the `#ifdef TEST_X` directives.
- This triggers an 'unused variable' in the function of Task 4, but you can ignore this warning. The executable `test_X` is generated and you can run it and compare the results with what we report above.

Definition of $bp()$

- For a given **int** a , we specify its binary representation by $(a_k, \dots, a_1, a_0)$. We denote by $bp(a)$ the bit positions that are turned on in the representation of a : $bp(a) = \{i | a_i = 1\}$.
For instance, the binary representation of $a = 4$ is $(1,0,0)$, and for $a = 13$ it is $(1,1,0,1)$.
The bit positions that are turned on in the binary representations are as follows:

$$bp(4) = \{2\} \text{ and } bp(13) = \{0,2,3\}.$$

Task 4

Complete the implementation of the function **getNextBitpos**, such that it returns the next position in $bp(a)$. The first time that function **getNextBitPos** is called with a given argument, it should return the LSB. Then, in every subsequent call to the function (with the same argument), it should return the next position in $bp(a)$, until it reaches the MSB. If the function is called once again with the same argument after the MSB is reached, it returns **-1**. if it is called another time then it returns the LSB, and so on. If the argument is switched at any point, then the function “resets” and returns the LSB of the current argument – the new integer number

Execution examples:

- **getNextBitpos(44)** returns 2
- **getNextBitpos(44)** returns 3
- **getNextBitpos(44)** returns 5
- **getNextBitpos(44)** returns -1
- **getNextBitpos(44)** returns 2
- **getNextBitpos(44)** returns 3
- **getNextBitpos(6)** returns 1

getNextBitpos(44)	returns 2
getNextBitpos(6)	returns 1
getNextBitpos(6)	returns 2
getNextBitpos(6)	returns -1

Recall;

44 $\rightarrow$ binary $\rightarrow$ 101100	$bp(44) = \{2,3,5\}$
6 $\rightarrow$ binary $\rightarrow$ 110	$bp(6) = \{1,2\}$

Static variables (`static`)

Local (`auto`) variable:

- scope confined to current code block (and all blocks contained in it)
- life span confined to current invocation of code (re-allocated every time block is entered)

Global (`extern`) variable:

- scope from definition point to end of code file
- life span is entire run time of program

Static (`static`) local variable:

- scope confined to current code block (and all blocks contained in it)
- life span is entire run time of program

Note: `static` in Java has a different meaning than in C++!!!

! Unlike Java !

Static variables (static) - Example

```
int fun()  
{  
    static int x = 5;  
    x++;  
    return x;  
}
```

Execution examples:

fun()	returns 6
fun()	returns 7
fun()	returns 8
fun()	returns 9
....	

When we call the function **fun** for the first time we initialize the variable **x** to be 5, next we increase its value by one and return 6.

For the next calls (second, third, fourth ..., and so on), the value of **x** does not re-initialize, Its value continues from the previous call.

default initialization value of a static variable is 0.

Task 5

Complete the implementation of the function `printBP`, such that it prints all positions of `bp(a)`. The printing format should be exactly as specified in the examples below, with a single space character between every two positions and a newline printed after the MSB position. You should modify the code of the function only between the two comment lines.

- **`printBP(44)`** prints: `bp(44) = 2 3 5`
- **`printBP(6)`** prints: `bp(6) = 1 2`
- **`printBP(0)`** prints: `bp(0) = (there is no space after =)`
- **`printBP(-1)`** prints: `bp(-1) = 0 1 2 3 . . . 30 31`